Dante-BabelBox user guide

Dante-BabelBox is a cross-vendor Dante control bridge: it translates between manufacturers' control protocols, so you can set preamp gain and phantom power and read wireless-mic status across consoles, stageboxes and receivers that don't otherwise talk to each other.

Dante carries your audio and basic device discovery, and there it stops. Preamp gain, phantom power and wireless-mic status are left to each vendor, and each one runs its own proprietary control protocol over the same network, none of which understands the others. Hence the name: this does the translating, so the state of one vendor's device can be used from outside that vendor's own kit.

It does two separate jobs:

- **Preamp control:** gain and phantom power bridged across consoles and stageboxes from different vendors, with a patch-bay web UI.
- **Radio-mic telemetry:** battery, RF and audio level from wireless receivers, across vendors, **whether or not the hardware has a Dante audio option installed**, because the adapters only ever talk to a device's IP control channel and never touch Dante audio at all.

> **Before you rely on this:** the adapters are built against official or community-authoritative vendor protocol specs rather than guesswork, and tested against mock devices. But **no adapter has been validated against real hardware**, and the **Lectrosonics mic adapter's wire format is an unverified placeholder**, so it's a scaffold rather than a working adapter.
>
> **One thing has been proven on hardware**, and it's worth knowing which. The Yamaha R-series HA protocol was captured from a real QL1 + Rio3224-D2, decoded, documented, rebuilt from that documentation and sent back, and the stagebox accepted it and changed its gain. That proves the *protocol* on hardware, but not any *code*: the write came from a standalone script, and the Rio adapter is still unimplemented. See yamaha-ha-remote-over-dante.md.
>
> This codebase was created with AI assistance, directed and reviewed by a human author.

Which devices are supported

Preamp control

Vendor	Device	Protocol	Status
Behringer / Midas	X32, M32	OSC	Done
Behringer	Wing	OSC	Done — **8 built-in preamps only**
Allen & Heath	AHM-series	NRPN over TCP	Done
Allen & Heath	dLive	NRPN over TCP (Socket addressing)	Done
Yamaha	DM3 / DM3S	OSC	Done
Allen & Heath	Qu, SQ	—	**Not implemented** — no public preamp-control spec exists
Yamaha	DM7, CL, QL	—	**Not implemented** — no public spec
Yamaha	Rio, Tio	Legacy AD8HR (MIDI SysEx)	**Not implemented** — setup docs exist, wire format doesn't

Radio-mic telemetry

Vendor	Device	Protocol	Status
Shure	ULX-D	ASCII over TCP 2202	Done
Shure	Axient Digital	ASCII over TCP 2202	Wire framing done; field behaviour only spot-checked
Sennheiser	EW-DX EM 2 / EM 2 Dante / EM 4 Dante	SSC (JSON over UDP)	Done
Lectrosonics	DSQD / Duet	Ethernet control port	**Placeholder wire format — correct the bytes and the port before use**

The Lectrosonics adapter still won't make anything up in what it *reports* (audio dBFS and RF dBm stay unset rather than being invented), but treat it as a scaffold until the wire format is confirmed.

Getting a bridge running

1. See what's on the network.

```
cargo run --bin preamp-bridge -- discover
```

This browses Dante's own mDNS advertisements. It tells you what's there. Whether this bridge can drive it is another matter.

2. Generate a config.

```
cargo run --bin preamp-bridge -- init --output bridge.toml
```

`init` discovers devices and probes each one against every implemented adapter's `identify()` until one claims it. **How sure it is depends on the protocol, and it's worth knowing which you're getting:**

- **X32-family and Wing** confirm vendor *and* model, from documented replies.
- **AHM and dLive** confirm the protocol family only, because their specs have no model-string query.
- **DM3 is the weakest signal**: its spec documents no identify query at all, so a scene-status request is reused as a presence probe.

`[[mapping]]` **entries aren't generated by default.** Add them by hand, or see below.

3. Run it.

```
cargo run --bin preamp-bridge -- run --config bridge.toml
```

Editing `bridge.toml` while the bridge runs **hot-reloads the mapping table**.

Inferring mappings from live Dante patching

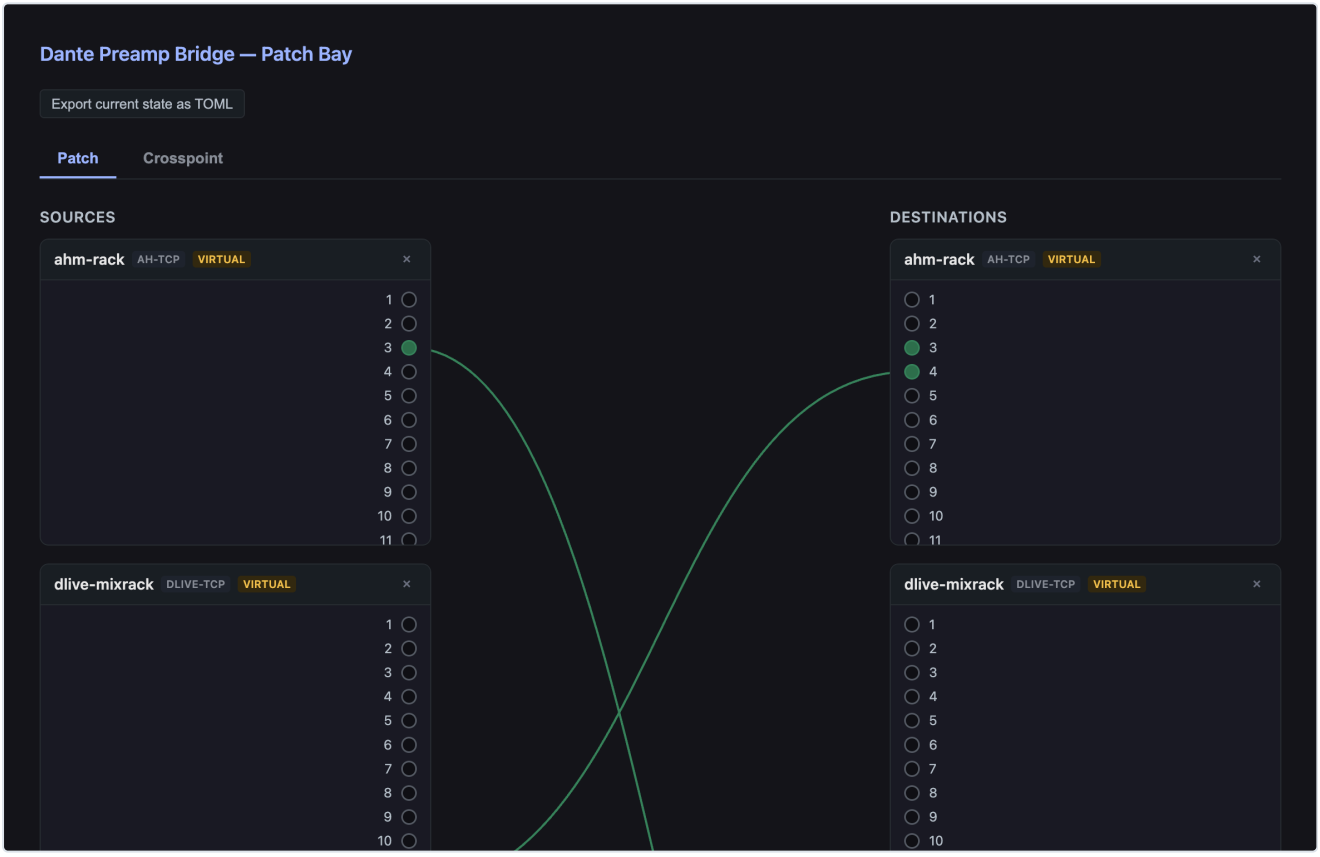
```
cargo run --bin preamp-bridge -- init --output bridge.toml --infer-mappings
```

With this flag, `init` also asks each identified device for its current Dante RX subscriptions, and writes a `[[mapping]]` for every subscription that points at another device in the file. It's a proper signal, read from live patching rather than guessed.

> **One catch, and it's why this is opt-in.** The channel numbers it writes are **Dante audio channel numbers**. On most gear those match the preamp/headamp channel a vendor adapter addresses, because out of the box Dante channel order mirrors physical I/O order 1:1. That's a *default-configuration convention*, **not a protocol guarantee**, and a console with customised Dante patching breaks the assumption without telling you.
>
> Treat inferred mappings as a first draft to check against each adapter's channel numbering.

The patch-bay web UI

`run` also serves a web UI, bound by default to `0.0.0.0:8080`, so any browser on the LAN can reach it at `http://<this-machine's-IP>:8080`.



*An actual screenshot of the running web UI. The devices are **virtual** placeholders, so the UI can be shown with no hardware on the network.*

- **Patch tab:** click a source channel, then a destination channel, to connect them. Click a patch cable to disconnect.
- **Crosspoint tab:** the same mappings as a matrix, showing only channels that are actually mapped. A full grid of every channel across even two devices would run to tens of thousands of cells, so pin a device to bring all its channels into the grid when you need them.

- **Gain lock tab:** when `bridge.toml` has a `[gain_lock]` table, this is where you elect the leader desk, lock and unlock, set the leader's preamp gains, and see what the lock has done to each follower's trim. See Gain compensation below.
- **Device management:** add real devices (they connect straight away) or **virtual** ones.
- **Export as TOML:** devices and mappings added through the UI are **in memory only and don't survive a restart**. This button exports the current state so you can paste it into `bridge.toml` and keep it.

Change the bind with `--web-bind 127.0.0.1:8080` to restrict it to this machine, or turn it off with `--no-web`.

> **There's no authentication and no TLS.** It's the same trust model as a hardware router's control port, so it belongs on a trusted operations network, not the open internet.

What "virtual" means

A virtual device is a placeholder for an **emulation layer that isn't built yet**. Today the bridge is a translating router you set up by IP and channel. That's useful, but the consoles can still tell it's there. Making it answer as if it *is* a native device of a foreign brand, so a console's own preamp UI drives it directly, needs packet captures of real gear paired with its own native device, and none of that handshake is in any public spec.

Virtual devices let you design the topology you want now, before there's any emulation to back it. If you can help with those captures, `docs/` has a non-technical field guide, one edition per OS: Windows · macOS · Linux.

The config file

```
[[device]]
id = "ahm-rack"
kind = "ah-tcp"        # osc-x32 | osc-wing | ah-tcp | dlive-tcp | yamaha-dm3
address = "10.0.0.10"
port = 51325           # optional — defaults to the protocol's standard port

[[device]]
id = "future-x32"
kind = "osc-x32"       # the protocol this virtual device will eventually emulate
virtual = true         # placeholder — no address needed
channels = 8

[[mapping]]
from = { device = "ahm-rack", channel = 3 }
to   = { device = "x32-monitors", channel = 7 }
bidirectional = true
```

> `channel` **means whatever the underlying protocol's own addressing unit is:** an X32 headamp number (1–128: local 1–32, AES50-A 33–80, AES50-B 81–128), a dLive physical preamp **Socket** number (which is *not* the processing channel number), or a DM3 Local Input number. It's the thing you're most likely to get wrong. Check the relevant adapter's module doc comment for the exact ranges.

`kind` is an open string rather than a fixed list, so a loaded plugin can register more kinds. There's a fully worked example in `bridge.example.toml`.

Gain compensation (the gain lock)

When several desks share one stage box, one desk owns the preamps, and every gain change it makes turns up on every other desk too, usually without warning. Declare that desk the **leader**, name the **signals** the desks share, and **lock**. From then on each follower desk's digital trim takes the opposite of every leader gain change, so the followers' levels stay put.

```
[gain_lock]
leader = "foh"

[[gain_lock.signal]]
name = "Kick In"
members = [
  { device = "foh", preamp = 1 },   # the leader's preamp
  { device = "mon", input = 1 },    # the follower's input channel, whose trim moves
]
```

What to know before you use it at a show:

- **Moves are relative.** The bridge reads each follower's live trim and moves it from there, so a trim change the monitor engineer makes mid-show is kept.
- **Nothing is lost at a limit.** A trim that runs out of range keeps the difference and takes it back when the leader comes down. A desk that drops off the network gets the missed amount when it comes back.
- **Followers today are X32/M32 and Wing.** They're the only protocols here that document a writable channel trim. Any device with a preamp can lead. On an X32 the documented trim covers digital sources only, so check it on a headamp-fed channel first.
- **Tested against mock desks only.** No real desk has followed a leader yet.

The full guide, with the rules, the desk survey and a first-rig checklist, is GAIN-LOCK.md.

Radio-mic telemetry

This is a separate binary, because the domain is a different shape. Telemetry is mostly reading (battery, RF and audio level are monitoring only, and mute is the only realistic write), so it has its own adapter trait rather than extending the preamp router.

```
cargo run --bin mic-monitor -- discover
cargo run --bin mic-monitor -- watch
```

mic-telemetry-architecture.md diagrams both vendors' wire protocols sequence by sequence, and shows what ends up in the reported state for each vendor.

Adding a device the bridge doesn't support

Device support loads two ways, and the router and the web UI can't tell them apart. The one that matters to you is **dynamic loading**: a plugin is a separate `.so` / `.dylib` / `.dll`, picked up from `--plugins-dir` (default `plugins`) at startup.

Adding a vendor needs no recompile of this project at all. Build your plugin, drop the file in and restart the bridge. All five real preamp vendors already work this way.

plugin-development-guide.md has the full contract, two worked examples, and the pitfalls that turned up while building it.

Troubleshooting

Symptom	Cause
`discover` lists devices, `init` doesn't claim them	Discovery is Dante's mDNS; identification is per-adapter. A device can be visible and unsupported.
`init` identified a DM3 that isn't one	DM3 has no identify query; a scene-status request stands in as a presence probe. It is the weakest signal of the five.
Gain moves the wrong channel	Channel numbering is per-protocol. On dLive especially, the Socket number is not the processing channel.
Inferred mappings are off by a channel or scrambled	The console has customised Dante patching, so Dante channel order no longer mirrors physical I/O.
Devices added in the web UI vanished after a restart	They are in-memory only. Use **Export as TOML** and keep it in `bridge.toml`.
A command bounces back and forth between two devices	Shouldn't happen — the router has echo suppression on bidirectional mappings. Worth reporting.
Web UI unreachable from another machine	`--web-bind` is set to loopback, or `--no-web` is on.
Wing preamps above 8 don't respond	Only the 8 built-in preamps are covered.
Lectrosonics receiver reports nothing	Expected — the wire format is an unverified placeholder.

See also

- plugin-development-guide.md: adding a device without a PR
- mic-telemetry-architecture.md: the telemetry domain in full
- yamaha-ha-remote-over-dante.md: the protocol note proven against real hardware
- API.md · DEVELOPING.md
- Capture guides for helping build device emulation: Windows · macOS · Linux
- README: status tables, architecture and downloads